Open in Colab

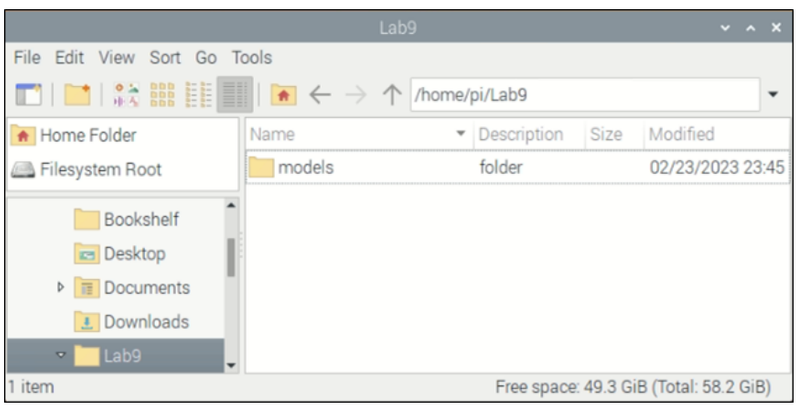
Lab 9.2 Machine Learning 2 - Machine Learning Implementation to Edge Device

TensorFlow implementation to Raspberry Pi

2.1 Load and Run TensorFlow Model

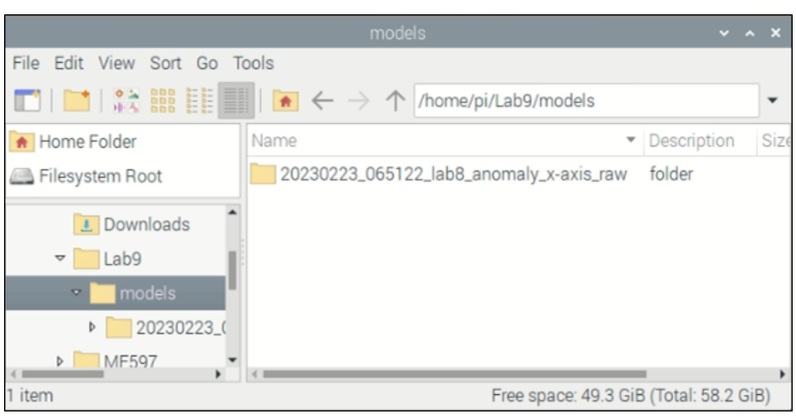
To implement the TensorFlow model which you developed and finally was selected in Lab 8 to Raspberry Pi, please follow the steps below. Please note that all required functions and the basic structure of the script are prepared in [lab9_ML_implementation.py](). 1. Create a working directory on Raspberry Pi.

a. In this examle, the directory is '/home/pi/Lab9'. Your directory may be different.

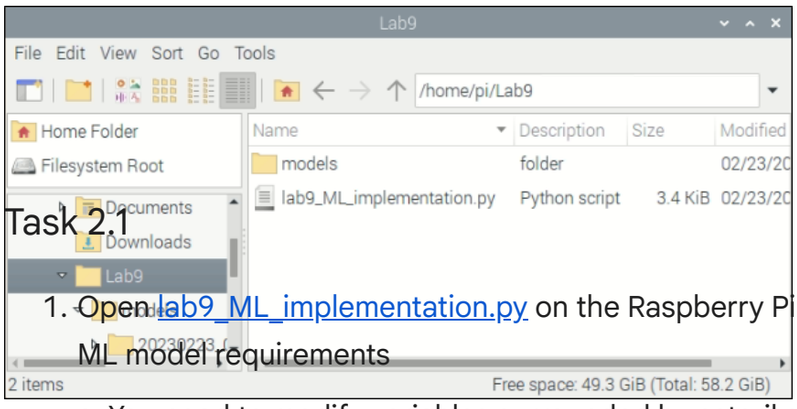


2. Copy and paste the trained TensorFlow model directory under '/models' directory of your working directory from laptop.

   a. In this example, the model directory is '/models/20230223_064122_lab8_anomaly_x-axis_raw'. Your directory must be different.



3. Copy and paste 'lab9_ML_implementation.py' to the same directory of Raspberry Pi as below.



Task 2.1

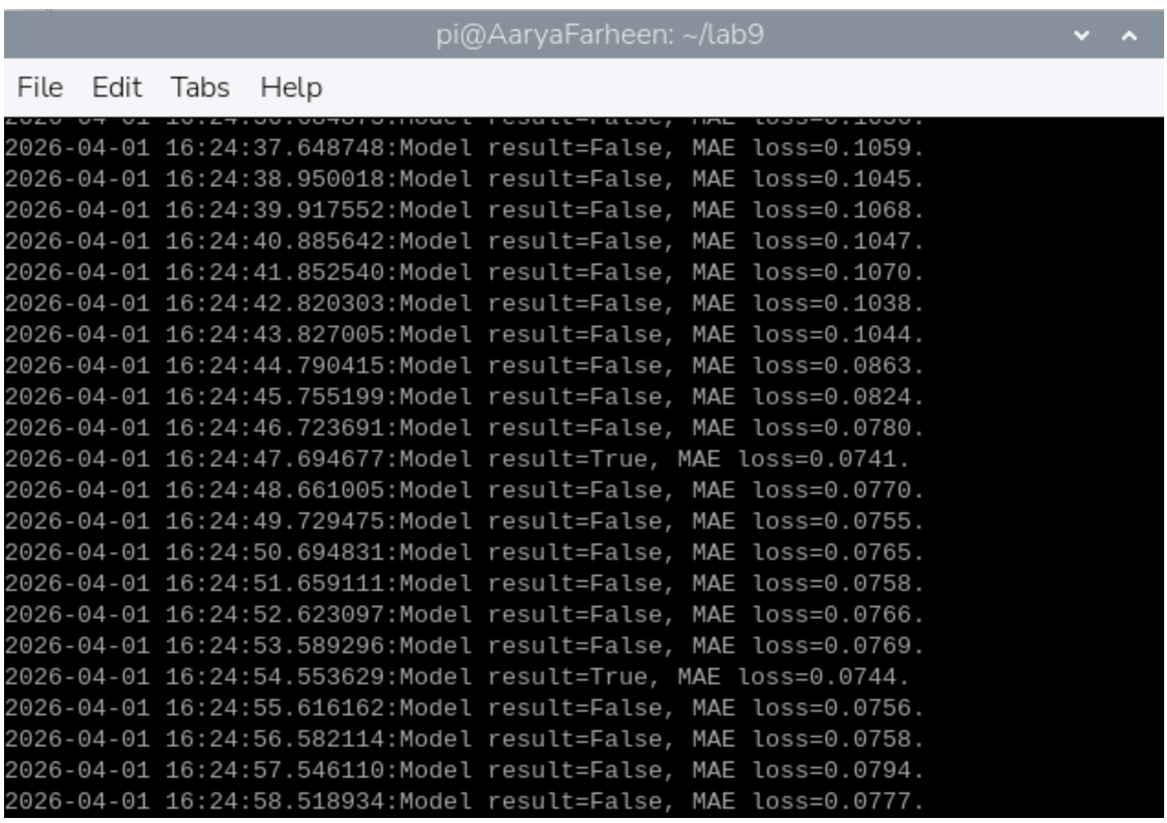
1. Open lab9_ML_implementation.py on the Raspberry Pi and then modify it according to your ML model requirements

   a. You need to modify variables surrounded by asteriks in the part of the code below. E.g., `*example_var* = /sample/dir/` -> `example_var = /real/dir/`.

Perform Task 2.1 to get the prediction result from the model and Task 2.2 to complete the algorithm (Figure 2).

2. Run the modified code, capture 'Terminal' or 'Thonny Shell' as Figure 5 and then attach it to the report.

   a. By turning the AFF on and off repeatedly, confirm if your model works well.



Python - Python3 ("Main" of lab9_ML_implementation.py)

```
if __name__ == "__main__":
    *model_path* = "models/YourModelDirectory/" # model file directory, you must change this
    model = tf.keras.models.load_model(model_path) # load the model from the path above
    *threshold* = # threshold (MAE loss) for the ML model
    *min_val* = # minimum value for normalization
    *max_val* = # maximum value for normalization

    while True:
        try:
            x, y, z = measureData(acc, 1000) # x=x-axis, y=y-axis, z-axis acceleration array
            *input_feature* = # your input feature
            input_feature_normalized = tensorNormalization(input_feature, min_val, max_val)
            result = predict(model, *FINAL_FEATURE_INPUT*, threshold) # your model result: b
            now = datetime.datetime.now() # datetime now
            print("{0}:Model result={1}, MAE loss={2:.4f}.".format(now, result[0], result[1]
        except KeyboardInterrupt:
```

```
            # to halt, press Ctrl + c
            raise
```

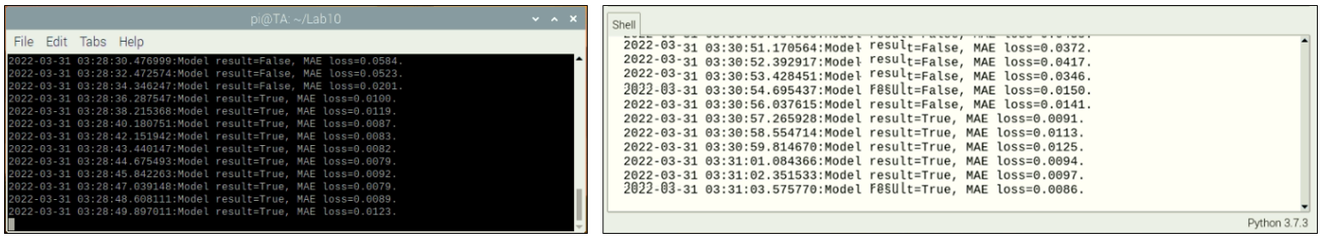


Figure 5 The result of the ML model by Task 2.1: 'Terminal' (left) and 'Thonny' (right)

Task 2.2 Answer to the questions below regarding Task 2.1:

1. Does the model work as expected?

Yes, the model works as expected. It successfully identifies whether the fan is ON or OFF based on the input data.

---

2. What does the result of the model mean?

The model result represents whether the system is operating normally or not. A True result indicates normal (running) operation, while a False result indicates abnormal or stopped condition based on the MAE loss compared to the threshold.

---

3. When the AFF stop, what is the result of the model?

When the AFF is stopped, the model outputs False because the MAE loss exceeds the threshold, indicating a non-running or abnormal condition.

Task 2.3

1. To complete algorithm as Figure 2, modify the code ('lab9_ML_implementation.py') after performing Task 1.1, Task 1.3, and Task 2.1.

a. When the AFF is running, the execution must be 'ACTIVE'.

b. When the AFF is running without the attached putty, the condition must be 'NORMAL'.

c. When the AFF is running with attached putty, the condition must be 'ABNORMAL'.

```
* **An alternative to make abnormal condition is tapping the AFF. It does make sense that ta
```

d. When the AFF is NOT running, the execution and the condition must be 'STOPPED' and 'UNAVAILABLE', respectively.

e. Please note that the algorithm can be expressed as below:

```
if AFF is running:
  execution = 'ACTIVE'
  if AFF is normal:
    condition = 'NORMAL'
```

```
  else:
    condition = 'ABNORMAL'
else:
  execution = 'STOPPED'
  condition = 'UNAVAILABLE'
```

2. The example result is shown in Figure 6.

a. If you are not sure about how to determine the execution, review [Task 1.1](#).

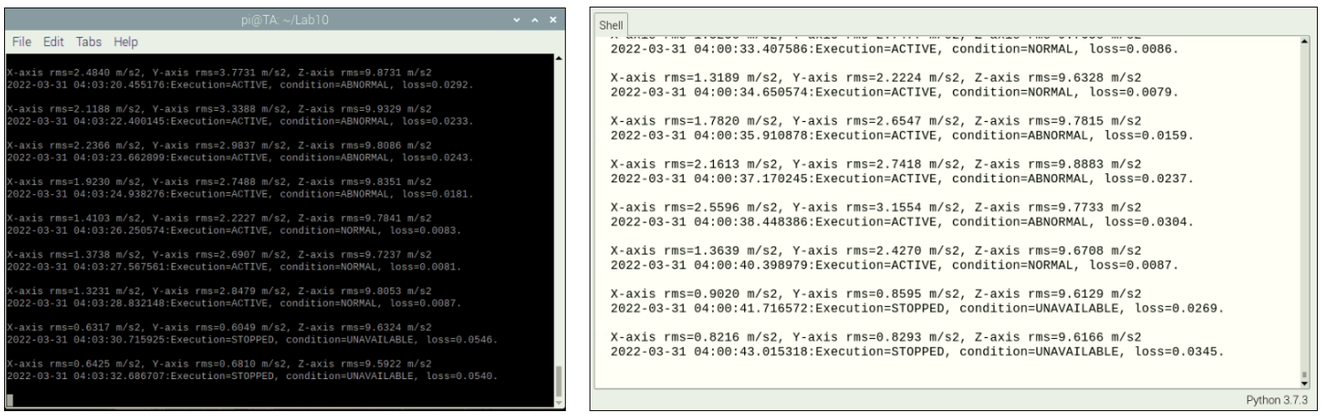
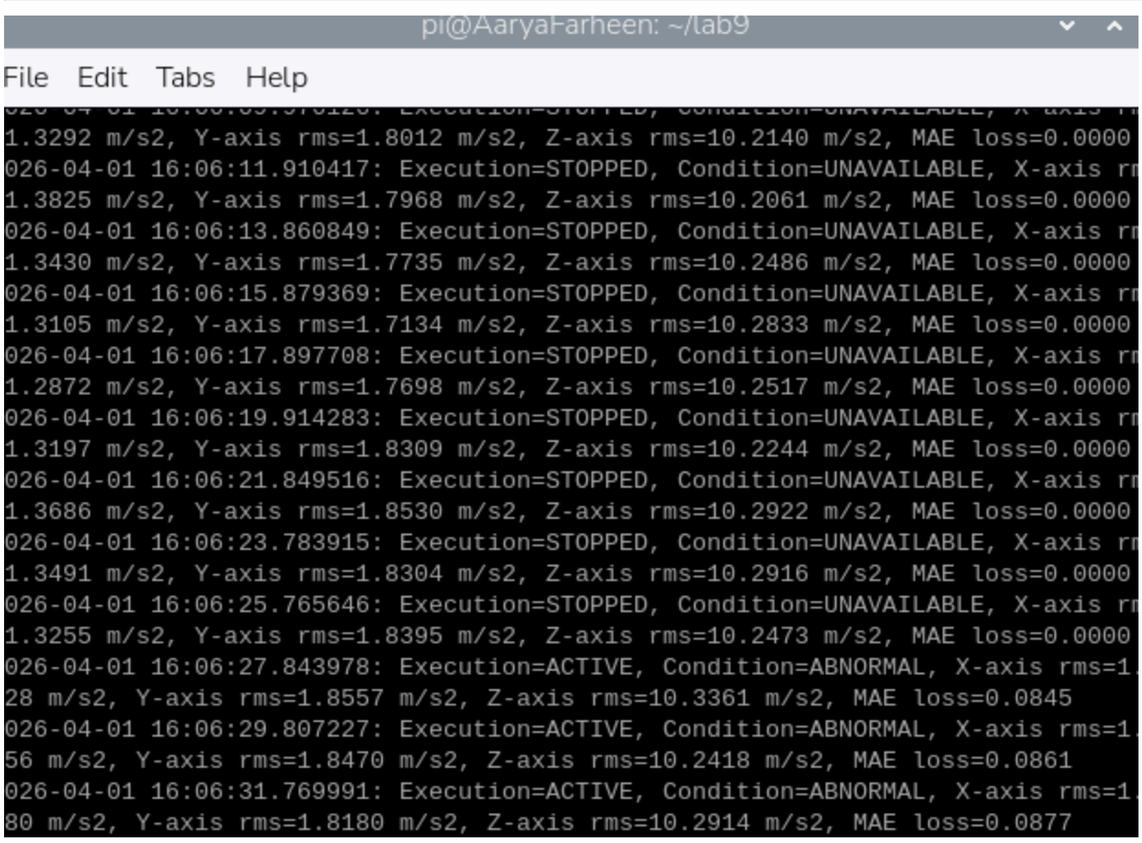


Figure 6 The result of algorithm by TASK4: 'Terminal' (left) and 'Thonny' (right)

Axis	STOPPED RMS Range (m/s²)	ACTIVE (Normal) RMS Range (m/s²)	ACTIVE (Abnormal) RMS Range (m/s²)	Separatio
X	0.857 – 0.987	1.823 – 2.129	2.903 – 3.127	Very High
Y	0.962 – 1.104	1.201 – 2.191	2.987 – 3.313	Moderate
Z	10.202 – 10.325	10.077 – 10.257	10.238 – 10.391	Very Low

The X-axis should be used to determine whether the AFF is running. The RMS threshold is approximately 1.41 m/s². If the X-axis RMS > 1.41 m/s², the AFF is considered running (ACTIVE). If the X-axis RMS ≤ 1.41 m/s², the AFF is considered stopped.



Task 2.4 Do your model and algorithm work as expected? Try change the running states (executions) and conditions.

- If yes, please explain your test setup and the results.
- If not, please explain the possible reasons and how to make it work.

Yes, the model and algorithm worked as expected. The test was performed by changing both the execution state and the operating condition of the AFF. First, when the AFF was not running, the X-axis RMS remained below the execution threshold, so the algorithm correctly identified the execution as STOPPED and the condition as UNAVAILABLE. Next, when the AFF was running normally without attached putty, the RMS exceeded the threshold and the execution was correctly identified as ACTIVE, while the anomaly detection model classified the condition as NORMAL. Finally, when the AFF was running with abnormal behavior, such as attached putty or tapping, the execution was still detected as ACTIVE, but the MAE loss increased beyond the threshold, so the condition was correctly classified as ABNORMAL. These results show that the algorithm can successfully distinguish among stopped, normal running, and abnormal running states.

Please continue to Lab 9.3 here.